

## 3.4.1 Variables and Data Types

ویری ایبل (Variable) میموری میں ایک نام والی جگہ ہے جس میں ڈیٹا رکھا جاتا ہے، بالکل ایک لیبل لگے ڈیے کی طرح۔

CS XI

Chapter 3: Programming Fundamentals

Lecture 05 of 13

★ Golden

### 3.4.1 Variables and Data Types ★ Golden

#### LEARN IT

A **variable** is a **named space in computer memory used to store data**. Think of it as a labelled container (a box with a name written on it). You put a value in it with the **assignment operator** `=`, which means "store", not "equals".

In Python you **do not need to declare the type** of a variable. Python figures it out automatically from the value. This is called **dynamic typing**: the same variable can later hold a different type.

#### Rules for naming variables:

- Can contain **letters, numbers and underscores** (`_`) only — no spaces or symbols like `-`, `@`, `$`.
- **Must start with a letter or underscore**, NOT a number (`1name` is invalid).
- Are **case-sensitive**: `age` and `Age` are two completely different variables.
- Cannot be a Python **keyword** (a reserved word such as `if`, `for`, `while`, `else`, `True`).

#### Valid names ✓

```
name = "Ali"
age = 16
my_height = 5.7
score1 = 100
```

#### Invalid names ✗

```
1name = "Ali" # starts with a number
my height = 5.7 # contains a space
if = 100 # keyword used
```

#### WORKED EXAMPLE

#### Using variables and case-sensitivity:

```
name = "Ali"
age = 16
my_height = 5.7
score1 = 100
print(name, age, my_height, score1)

Age = 17 # a different variable from age
print(age, Age)
```

#### OUTPUT

```
Ali 16 5.7 100
16 17
```

**Dynamic typing** — the type follows the value. `type()` tells you the type:

```
x = 5
print(x, type(x))
x = "five"
print(x, type(x))
```

```
OUTPUT
5 <class 'int'>
five <class 'str'>
```

#### What Python says about an invalid name:

```
1name = "Ali"
```

```
OUTPUT
File "names.py", line 1
    1name = "Ali"
      ^
SyntaxError: invalid decimal literal
```

#### REMEMBER IT

Choose **meaningful names**: `total_marks` is far clearer than `tm` or `x`. Use lowercase words joined by underscores (this style is called **snake\_case**).

#### COMMON MISTAKE

`=` is **not** "equals" in the maths sense. `x = x + 1` is not a false equation; it means "take the value of `x`, add 1, and store the result back in `x`". To **compare** two values you use `==` (see 3.4.3).

#### اردو میں سمجھیں

ویری ایبل (Variable) میموری میں ایک نام والی جگہ ہے جس میں ڈیٹا رکھا جاتا ہے، بالکل ایک لیبل لگے ڈیے کی طرح۔ پائتھن میں ٹائپ بتانے کی ضرورت نہیں ہوتی۔ نام حرف یا انڈر سکور سے شروع ہونا چاہیے، اس میں خالی جگہ نہیں ہو سکتی، یہ کیس سینسیٹو (Case-sensitive) ہوتا ہے اور کوئی کی ورڈ (Keyword) نہیں ہو سکتا۔

#### EXAM TIP

A favourite question: "Which of these variable names are valid? Give reasons." Check each name against all four rules and state the rule broken, e.g. `2nd_place` — invalid, starts with a digit". Remember that `While` (capital W) is **valid** because keywords are lowercase and Python is case-sensitive.

#### CHECK YOURSELF

1. Valid or invalid? `_total`, `roll-no`, `class`, `marks2026`, `2marks`
2. Predict the output:

```
city = "Karachi"
City = "Hyderabad"
print(city)
```

Answers: 1. `_total` valid; `roll-no` invalid (hyphen); `class` invalid (keyword); `marks2026` valid; `2marks` invalid (starts with a digit). 2. `Karachi` — `city` and `City` are different variables.

## LEARN IT

A **data type** specifies the **kind of data** a variable holds — a whole number, a decimal, text, True/False, or a collection of items. The type decides what you can do with the data (you can divide numbers, but not names). Python has several built-in data types:

| Name           | Type               | Description                                                             | Example                                                   |
|----------------|--------------------|-------------------------------------------------------------------------|-----------------------------------------------------------|
| Integer        | <code>int</code>   | Whole numbers without decimals (positive, negative or zero).            | <code>age = 16</code>                                     |
| Floating point | <code>float</code> | Numbers with a decimal point.                                           | <code>height = 5.7</code>                                 |
| String         | <code>str</code>   | Text data, enclosed in quotes ( " " or ' ' ).                           | <code>name = "Sammy"</code> or <code>year = "2026"</code> |
| Boolean        | <code>bool</code>  | Logical value: <code>True</code> or <code>False</code> (capital T / F). | <code>is_student = True</code>                            |
| List           | <code>list</code>  | Ordered, changeable collection in square brackets [ ].                  | <code>marks = [85, 90, 75]</code>                         |
| Tuple          | <code>tuple</code> | Ordered, unchangeable sequence in parentheses ( ).                      | <code>coords = (10, 20)</code>                            |
| Dictionary     | <code>dict</code>  | Key : value pairs in curly braces { }.                                  | <code>student = {"name": "Ahmed", "age": 16}</code>       |
| Set            | <code>set</code>   | Unordered collection of unique items (no duplicates).                   | <code>vowels = {'a', 'e', 'i'}</code>                     |

Note: the slide abbreviates tuple as "tup"; the real Python type name is `tuple`, as the program below shows. The slides describe dictionaries as unordered; since Python 3.7 a dictionary remembers the order in which keys were added, but you always look items up **by key**, not by position.

## WORKED EXAMPLE

Checking every type with `type()` :

```
age = 16
height = 5.7
name = "Sammy"
year = "2026"
is_student = True
marks = [85, 90, 75]
coords = (10, 20)
student = {"name": "Ahmed", "age": 16}
vowels = {'a', 'e', 'i'}

print(type(age))
print(type(height))
print(type(name), type(year))
print(type(is_student))
print(type(marks))
print(type(coords))
print(type(student))
print(type(vowels))
```

#### OUTPUT

```
<class 'int'>
<class 'float'>
<class 'str'> <class 'str'>
<class 'bool'>
<class 'list'>
<class 'tuple'>
<class 'dict'>
<class 'set'>
```

Notice that `year = "2026"` is a **string**, not a number, because it is inside quotes.

#### Working with the collection types:

```
marks = [85, 90, 75]
marks[0] = 88          # lists are changeable
marks.append(60)
print(marks)

coords = (10, 20)
print(coords[0])       # read item at position 0

student = {"name": "Ahmed", "age": 16}
print(student["name"]) # look up by key

nums = {1, 2, 2, 3, 3, 3} # duplicates are removed
print(nums)
print(len(nums))
```

#### OUTPUT

```
[88, 90, 75, 60]
10
Ahmed
{1, 2, 3}
3
```

Positions (called **indexes**) start at **0**, so `coords[0]` is the first item. Trying to change a tuple causes an error:

```
coords = (10, 20)
coords[0] = 50
```

#### OUTPUT

```
Traceback (most recent call last):
  File "tuple_test.py", line 2, in <module>
    coords[0] = 50
    ~~~~~^
TypeError: 'tuple' object does not support item assignment
```

#### REMEMBER IT

**Brackets decide the collection:** `[]` List (changeable), `()` Tuple (locked), `{ }` with **key:value** = Dictionary, `{ }` with single items = Set (unique). Think: a **list** is a shopping list you can edit; a **tuple** is your date of birth — it never changes.

### COMMON MISTAKE

`True` and `False` must start with a capital letter. `true` is not a Boolean; Python treats it as an unknown variable name and gives a `NameError`. Also, `"16"` (string) and `16` (int) are different: `"16" + "1"` gives `"161"`, while `16 + 1` gives `17`.

### اردو میں سمجھیں

ڈیٹا ٹائپ (Data Type) بتاتی ہے کہ ویری ایبل میں کس قسم کا ڈیٹا ہے۔ `int` مکمل اعداد، `float` اعشاریہ والے اعداد، `str` متن، اور `bool` صرف `True` یا `False` کے لیے ہے۔ لسٹ (List) تبدیل ہو سکتی ہے، ٹپل (Tuple) تبدیل نہیں ہو سکتا، ڈکشنری (Dictionary) میں کی اور ویلیو کے جوڑے ہوتے ہیں، اور سیٹ (Set) میں کوئی چیز دہرائی نہیں جاتی۔

### EXAM TIP

Expect: "Name any four data types with examples" or "Differentiate list and tuple". For a difference question, use a two-column table: list — `[ ]`, changeable, e.g. `marks = [85, 90]`; tuple — `( )`, unchangeable, e.g. `coords = (10, 20)`.

### CHECK YOURSELF

1. What is the data type of each? `3.0`, `"3"`, `3`, `False`, `[3]`
2. Predict the output: `print(len({5, 5, 6, 6, 6}))`
3. Which type should store a student's CNIC/B-Form number such as `"42101-1234567-1"`? Why?

Answers: 1. float, str, int, bool, list. 2. 2 (the set keeps only 5 and 6). 3. str, because it contains hyphens and you never do arithmetic on it.

## Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

**MCQ 1.** A variable is:

- a. A named memory space
- b. A printer
- c. An OSI hub
- d. A sprint

**MCQ 2.** Python types include:

- a. int, float, str, bool
- b. Only NAND
- c. Only charts
- d. Only files

**MCQ 3.** You need to declare types first in Python?

- a. No — assignment creates the variable
- b. Yes, like old C always
- c. Only for print
- d. Only in SQL

**MCQ 4.** True and False are:

- a. bool
- b. str
- c. float only
- d. lists

**MCQ 5.** "17" is a:

- a. str, not int
- b. int
- c. bool
- d. dict

**MCQ 6.** A legal name is:

- a. total\_marks
- b. 2nd
- c. class
- d. total-marks

## Extra short questions

1. Define variable and give the assignment syntax.
2. List four data types with one example each.
3. Why is "17" + "1" = "171"?
4. Give two identifier rules.

## Extra long question

1. A program reads two marks as input() and should print the average. Show types, conversions, and a common bug if you forget int().

## Extra answer key

**MCQs:** 1. A 2. A 3. A 4. A 5. A 6. A

### Short

1. Named storage. name = value.
2. int 5; float 5.0; str "Ali"; bool True.
3. Strings concatenate; convert with int() for arithmetic.
4. Start with letter/underscore; no spaces; no keywords; case-sensitive.

### Long

1. a=int(input()); b=int(input()); print((a+b)/2). Without int, + concatenates strings. Average is float.